



Taller 1: Definición recursiva de programas e inducción (Racket)

Fundamentos de Interpretación y Compilación de Lenguajes de Programación / Grupo 01 / Prof. Robinson Duque / Emily Nuñez / 2026-I

Evaluación por Indicadores - RES 047

Los indicadores de logro a evaluar en este Taller son:

- IL 1.1.1 Construye datos recursivamente utilizando métodos de inducción y gramáticas BNF (ejercicios 1 a 13)
- IL 1.1.2 Utiliza gramáticas BNF para guiar la construcción de programas recursivos (ejercicios 14 a 15)

Este taller tiene un valor global de 5.44% distribuidos en (3.2%) para el IL 1.1.1 y (2.24%) para el IL 1.1.2. El taller se calificará con una nota en escala de 0 a 100 puntos (que luego se escalará de 0.0 a 5.0). El valor de cada indicador se muestra a continuación:

$$IL1.1.1 = \frac{3.2 * 100}{5.44} = 59puntos(aprox)$$

$$IL1.1.2 = \frac{2.24 * 100}{5.44} = 41puntos(aprox)$$

Cada ejercicio será probado con un conjunto de pruebas unitarias que deberá pasar para asignar la totalidad de los puntos indicados. En caso de no pasar las pruebas, el valor a asignar será de 0.0pts. Asegurese de respetar el contrato de cada función.

Ejercicios - IL 1.1.1 (59pts)

1. **(4.5pts)** Elabore una función llamada *filter-pairs* que recibe como argumentos: una lista **L** y dos predicados **P** y **Q**. La lista **L** se compone de pares x, y que a su vez son listas (de tamaño 2). La función debe retornar una lista similar a **L**, que contenga únicamente aquellos pares (x, y) en los cuales x cumple con el predicado **P** y y cumple con el predicado **Q**. Los pares que no cumplan ambas condiciones deben ser descartados. *Ejemplos:*

```
> (filter-pairs '((3 6) (5 10) (8 12) (7 14)) odd? even?)  
((3 6) (5 10) (7 14))
```

```
> (filter-pairs '((5 9) (10 90) (82 7) (15 20)) even? even?)  
((10 90))
```

```
> (filter-pairs '((6 9) (10 90) (82 7)) odd? odd?)  
()
```

2. **(4.5pts)** Elabore una función llamada *transform-pairs* que recibe como argumentos: una lista **L**, un predicado **P** y una función **F**. La lista **L** se compone de pares x, y que a su vez son listas (de tamaño 2). La función debe retornar una lista similar a **L**. Para cada par (x, y) , si ambos elementos cumplen con el predicado **P**, se debe incluir en la lista resultante el par $(F(x), F(y))$. En caso contrario, se debe incluir el par original sin modificar. *Ejemplos:*

```
> (transform-pairs '((2 4) (3 6) (8 10)) even? add1)  
((3 5) (3 6) (9 11))
```

```
> (transform-pairs '((3 2) (4 2) (1 5) (2 8)) even? square)  
((3 2) (16 4) (1 5) (4 64))
```

```
> (transform-pairs '((2 6) (4 8) (5 10)) even? square)  
((4 36) (16 64) (5 10))
```

3. **(4.5pts)** Elabore una función llamada *flatten-one* que recibe como argumento una lista **L**. La función debe retornar una nueva lista eliminando exactamente un nivel de anidamiento de **L**. Los elementos que no sean listas deben conservarse sin modificación. *Ejemplos:*

```
> (flatten-one '((1 2) (3 4) (5 6)))
(1 2 3 4 5 6)

> (flatten-one '((una) (buena idea) ((de programacion))))
(una buena idea (de programacion))

> (flatten-one '(un (objeto (mas)) complicado))
(un objeto (mas) complicado)
```

3. **(4.5pts)** Elabore una función llamada *list-remove-if* que reciba tres argumentos: una lista **L**, un número **n** y un predicado **P**. La función debe retornar una lista similar a la que recibe (**L**), pero debe eliminar el elemento que se encuentra en la posición ingresada **n** (indexando desde cero) sólo si dicho elemento cumple con el predicado **P**. Si el elemento no cumple con el predicado, la lista debe permanecer sin modificaciones. *Ejemplos:*

```
> (list-remove-if '(5 8 7 6) 2 odd?)
(5 8 6)

> (list-remove-if '(5 8 7 6) 2 even?)
(5 8 7 6)

> (list-remove-if '(5 8 7 6) 3 mayor5?)
(5 8 7)

> (list-remove-if '(5 8 7 6) 0 mayor5?)
(8 7 6)
```

5. **(4.5pts)** Elabore una función llamada (**reverse-filter-equal? L1 L2 P**) que determine si dos listas contienen, en orden inverso, los mismos elementos que satisfacen un predicado. La función recibe como parámetros dos listas **L1** y **L2**, y un predicado **P**. La función debe considerar únicamente aquellos elementos de cada lista que satisfacen el predicado **P**. Retorna **#t** si, después de ignorar los elementos que no satisfacen el predicado, la segunda lista corresponde a la primera recorrida en orden inverso, y **#f** en caso contrario. La solución debe realizarse de manera recursiva y no se permite utilizar la función **reverse** ni funciones de filtrado de biblioteca.

Ejemplos:

```
> (reverse-filter-equal? '(1 2 3 4 5) '(5 3 1 8 6) odd?)  
#t
```

```
> (reverse-filter-equal? '(1 2 3 4 5) '(5 3 2 1) odd?)  
#t
```

```
> (reverse-filter-equal? '(1 2 3 4 5) '(5 3 2 1) even?)  
#f
```

```
> (reverse-filter-equal? '(a 2 b 4 c 6) '(8 c 10 b 12 a) symbol?)  
#t
```

6. (4.5pts) Elabore una función llamada *replace-nth* que recibe 4 argumentos: un elemento **E**, un elemento **R**, un número **N** y una lista **L**. La función debe retornar una lista similar a **L**, pero reemplazando por **R** únicamente la **N**-ésima ocurrencia de **E** en la lista, contando las ocurrencias desde cero. Si **E** aparece menos de **N**+1 veces en **L**, la lista debe retornar sin modificaciones. La solución debe realizarse de manera recursiva.

```
> (replace-nth 'a 'x 0 '(a b a c a))  
(x b a c a)
```

```
> (replace-nth 'a 'x 1 '(a b a c a))  
(a b x c a)
```

```
> (replace-nth 'a 'x 2 '(a b a c a))  
(a b a c x)
```

```
> (replace-nth 'a 'x 4 '(a b a c a))  
(a b a c a)
```

```
> (replace-nth 2 9 1 '(2 4 2 6 2))  
(2 4 9 6 2)
```

7. (4.5pts) Elabore una función llamada *cartesian-filter* que recibe como argumentos dos listas **L1** y **L2** y un predicado de dos argumentos **P**. La

función debe retornar una lista de tuplas que representen los pares posibles entre un elemento de **L1** y un elemento de **L2**, pero únicamente debe incluir aquellos pares para los cuales el predicado **P** retorne **#t**. Los pares pueden aparecer en cualquier orden. La solución debe realizarse de manera recursiva.

```
> (cartesian-filter '(1 2 3) '(4 5 6)
      (lambda (x y) (< x y)))
((1 4) (1 5) (1 6) (2 4) (2 5) (2 6) (3 4) (3 5) (3 6))

> (cartesian-filter '(1 2 3) '(1 2 3 4)
      (lambda (x y) (= (+ x y) 5)))
((1 4) (2 3) (3 2))

> (cartesian-filter '(a b c) '(x y)
      (lambda (x y) (not (eqv? x 'b))))
((a x) (a y) (c x) (c y))

> (cartesian-filter '(1 2 3 4) '(2 4 6)
      (lambda (x y) (= (modulo y x) 0)))
((1 2) (1 4) (1 6) (2 2) (2 4) (2 6) (3 6))
```

8. **(4.5pts)** Elabore una función llamada *group-by* que debe recibir como entrada 2 argumentos: una función unaria llamada **F** y una lista **L**. La función debe retornar una lista de pares de la forma **(v, L')** donde **v** es un valor producido por la función **F** y **L'** es una lista que contiene todos los elementos de **L** para los cuales la función **F** retorna dicho valor.

Los grupos deben aparecer en el orden en que se obtiene por primera vez cada valor de **F** al recorrer **L** de izquierda a derecha. Dentro de cada grupo, los elementos deben conservar el orden en que aparecen originalmente en **L**. Cada elemento de **L** debe pertenecer a exactamente un grupo.

Ejemplos:

```
> (group-by (lambda (x) (modulo x 2))
      '(1 2 3 4 5 6))

((1 (1 3 5)) (0 (2 4 6)))
```

```

> (group-by (lambda (x) (* x x))
            '(1 2 1 3 2))

((1 (1 1)) (4 (2 2)) (9 (3)))

> (group-by (lambda (x) (if (number? x) 'numero 'otro))
            '(a 2 b 4 c 7))

((otro (a b c)) (numero (2 4 7)))

```

9. **(4.5pts)** Elabore una función llamada *count-pairs* que recibe como entrada una lista de números **L** y un predicado binario **P**. La función debe retornar el número de parejas de elementos **(a,b)** de **L** que cumplen simultáneamente las siguientes condiciones: **a** debe aparecer antes que **b** en la lista y el predicado **P(a,b)** debe retornar **#t**. Cada pareja debe contarse una sola vez. *Ejemplos:*

```

> (count-pairs '(2 3 8 6 1)
    (lambda (a b) (> a b)))
5

> (count-pairs '(1 2 3 4)
    (lambda (a b) (> a b)))
0

> (count-pairs '(3 2 1)
    (lambda (a b) (> a b)))
3

> (count-pairs '(1 2 3 4 5)
    (lambda (a b) (= (+ a b) 6)))
2

```

10. **(4.5pts)** Elabore una función llamada (*alternating?* L P) que determine si una lista L cumple una condición de alternancia determinada por

un predicado binario P. La función recibe como parámetros una lista L y un predicado P. La función debe retornar **#t** si para cada pareja de elementos consecutivos de L, el predicado P aplicado a ambos elementos retorna **#t**. En caso contrario, debe retornar **#f**. La solución debe ser recursiva.

Ejemplos:

```
> (alternating? '(1 2 3 4 5)
      (lambda (a b) (< a b)))
#t
```

```
> (alternating? '(5 4 3 2 1)
      (lambda (a b) (> a b)))
#t
```

```
> (alternating? '(1 3 2 4 3)
      (lambda (a b) (< a b)))
#f
```

```
> (alternating? '(a b c d)
      (lambda (a b)
        (not (equiv? a b))))
#t
```

```
> (alternating? '(a a b c)
      (lambda (a b)
        (not (equiv? a b))))
#f
```

```
> (alternating? '()
      (lambda (a b) (< a b)))
#t
```

11. (4.5pts) Elabore una función llamada *merge-by* que recibe como entrada tres parámetros: una función binaria **F** y dos listas **L1** y **L2**, ambas de igual tamaño. La función **F** recibe como argumentos dos elementos, uno de cada lista, y debe retornar **#t** o **#f**. El procedimiento *merge-by* debe retornar

una nueva lista donde la posición n -ésima contiene el elemento de **L1** si **F** retorna **#t** al aplicarse sobre los elementos correspondientes de **L1** y **L2**; en caso contrario, debe contener el elemento de **L2**. *Ejemplos:*

```
> (merge-by > '(1 8 3 10) '(6 2 7 4))
(6 8 7 10)

> (merge-by < '(1 8 3 10) '(6 2 7 4))
(1 2 3 4)

> (merge-by = '(1 4 3) '(1 2 5))
(1 2 5)

> (merge-by (lambda (a b) (= (modulo a 2)
                              (modulo b 2)))
      '(1 4 7 8)
      '(2 6 3 5))
(2 4 7 5)
```

12. (4.5pts) Elabore una función llamada *filter-map-acum* que recibe como entrada 6 parámetros: dos números **a** y **b**, una función unaria **G**, una función binaria **F**, un valor inicial **acum** y un predicado **P**. El procedimiento *filter-map-acum* debe recorrer todos los números del intervalo $[a, b]$. Para cada número que satisfaga el predicado **P**, debe aplicar la función **G** sobre dicho número y utilizar el resultado como segundo argumento de la función binaria **F**, utilizando como primer argumento el valor actual de **acum**. El nuevo resultado debe convertirse en el valor actualizado de **acum**. Los elementos que no satisfagan el predicado deben ser ignorados. Finalmente, la función debe retornar el valor final de **acum**. *Ejemplos:*

```
> (filter-map-acum 1 5 (lambda (x) (* x x)) + 0 odd?)
35

> (filter-map-acum 1 5 (lambda (x) (* x 2)) + 0 even?)
12
```

```
> (filter-map-acum 1 4 (lambda (x) (+ x 1)) * 1 odd?)  
8
```

```
> (filter-map-acum 2 6 (lambda (x) (* x 10)) + 5 even?)  
125
```

13. (5pts) Elabore una función llamada (`operate lrators lrand`s) donde *lrators* es una lista de funciones binarias de tamaño n y *lrand*s es una lista de números de tamaño $n + 1$. La función retorna el resultado de aplicar sucesivamente las operaciones en *lrators* a los valores en *lrand*s.

```
> (operate (list + * + - *) '(1 2 8 4 11 6))  
102  
> (operate (list *) '(4 5))  
20
```

En el ejemplo anterior, el resultado es 102 puesto que $(((((1 + 2) * 8) + 4) - 11) * 6) = 102$ y 20 puesto que $(4 * 5) = 20$.

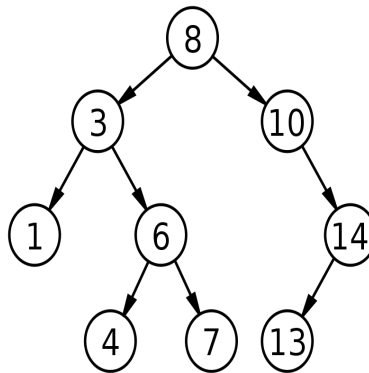
Ejercicios -IL 1.1.2 (41%)

14. (8pts) Elabore una función llamada *ancestors* que recibe como entrada dos parámetros: un número n y un árbol binario de búsqueda (representado con listas) **BS**T (el árbol debe contener el número entero n). La función debe retornar una lista con los valores de los nodos que se encuentran en la ruta desde el nodo raíz hasta el padre del nodo que contiene el número n , manteniendo el orden desde la raíz hasta dicho padre. Si el número n se encuentra en el nodo raíz, el procedimiento debe retornar una lista vacía.

Nota aclaratoria: Para el ejercicio se utiliza la representación de Árbol Binario de Búsqueda con Listas en Racket, y podría representarse con la ayuda de la siguiente gramática BNF:

```
<árbol-binario> := empty  
                := (número <árbol-binario> <árbol-binario>)
```

La propiedad de Árbol Binario de Búsqueda debe utilizarse para determinar si la búsqueda debe continuar por el subárbol izquierdo o por el subárbol derecho.



Es decir que este Árbol Binario de Búsqueda de la imagen, representado en Racket con listas y usando la anterior gramática, sería:

```
'(8 (3 (1 () ()) (6 (4 () ()) (7 () ()))) (10 () (14 (13 () ()) ())))
```

Ejemplos:

```
> (ancestors 13
```

```
  '(8 (3 (1 () ()) (6 (4 () ()) (7 () ())))
    (10 () (14 (13 () ()) ())))
```

```
(8 10 14)
```

```
> (ancestors 14
```

```
  '(8 (3 (1 () ()) (6 (4 () ()) (7 () ())))
    (10 () (14 (13 () ()) ())))
```

```
(8 10)
```

```
> (ancestors 4
```

```
  '(8 (3 (1 () ()) (6 (4 () ()) (7 () ())))
    (10 () (14 (13 () ()) ())))
```

```
(8 3 6)
```

15. (8pts) Elabore una función llamada *same-elements?* que recibe como entrada dos árboles binarios de búsqueda **BST1** y **BST2**, representados con listas. La función debe retornar **#t** si ambos árboles contienen exactamente los mismos números, sin importar que tengan estructuras diferentes, y **#f** en caso contrario. Se supone que los árboles no contienen elementos repetidos. La solución debe realizarse de manera recursiva.

Ejemplos:

```
> (same-elements?
  '(8 (3 (1 () ()) (6 (4 () ()) (7 () ())))
    (10 () (14 (13 () ()) ())))
  '(6 (3 (1 () ()) (4 () ()))
    (10 (8 (7 () ()) ())
      (14 (13 () ()) ())))
```

#t

```
> (same-elements?
  '(8 (3 (1 () ()) (6 (4 () ()) (7 () ())))
    (10 () (14 (13 () ()) ())))
  '(8 (3 (1 () ()) (6 (4 () ()) (7 () ())))
    (10 () (15 (13 () ()) ())))
```

#f

```
> (same-elements?
  '(5 (3 (1 () ()) (4 () ()))
    (8 (7 () ()) (10 () ())))
  '(8 (3 (1 () ()) (5 (4 () ()) (7 () ())))
    (10 () ())))
```

#t

16. (8pts) Elabore una función llamada (**caminos-suma arbol n**) que encuentre todos los caminos desde la raíz hasta una hoja de un árbol binario de búsqueda (BST) cuya suma de elementos sea igual a **n**.

La función debe retornar una lista con todos los caminos cuya suma sea exactamente **n**. Los caminos deben aparecer en orden de recorrido, explorando primero el subárbol izquierdo y posteriormente el subárbol derecho. Si no existe ningún camino que cumpla la condición, la función debe retornar la lista vacía. La función debe implementarse utilizando recursión y debe aprovechar la propiedad de orden del árbol binario de búsqueda cuando sea posible.

Ejemplos:

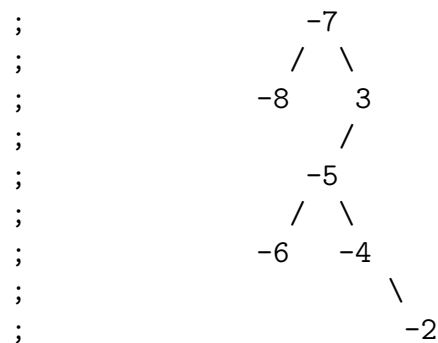
```
> (caminos-suma
    '(8 (3 (1 () ())) (6 (4 () ())) (7 () ())))
      (10 () (14 (13 () ())) ())))
12)
((8 3 1))
```

```
> (caminos-suma
    '(8 (3 (1 () ())) (6 (4 () ())) (7 () ())))
      (10 () (14 (13 () ())) ())))
21)
```

```
((8 3 6 4))
```

```
> (caminos-suma
    '(-7 (-8 () ()))
      (3 (-5 (-6 () ())) (-4 () (-2 () ()))) ()))
-15)
```

```
((-7 -8) (-7 3 -5 -6) (-7 3 -5 -4 -2))
```



```
> (caminos-suma
    '(8 (3 () (7 () ()))
        (10 () ()))
    18)
((8 3 7) (8 10))
```

17. **(8pts)** Elabore una función llamada `(coin-change monto monedas)` que calcule el número total de formas distintas de obtener un monto dado utilizando una lista de denominaciones de monedas. La función recibe como parámetros:

- **monto:** un número entero no negativo.
- **monedas:** una lista de números enteros positivos que representan las denominaciones disponibles.

La función debe retornar el número de combinaciones posibles para obtener exactamente el valor **monto**. La solución debe implementarse utilizando definición recursiva. *Ejemplos:*

```
> (coin-change 5 '(1 5))
2
> (coin-change 5 '(1 2 5))
4
> (coin-change 10 '(2 5 3 6))
5
```

18. **(9pts)** Elabore una función llamada `(to-infix E)` que reciba una expresión aritmética representada mediante listas en notación prefija y retorne una lista que represente la misma expresión en notación infija.

Las expresiones pueden contener los operadores `+`, `-`, `*` y `/`, además de números enteros.

Por ejemplo, la expresión:

$$(+ \ 2 \ (* \ 3 \ 4))$$

representa la expresión matemática $2 + (3 * 4)$.

La función debe convertir recursivamente cada subexpresión. Por ejemplo:

$$\begin{array}{c}
 (+\ 2\ (*\ 3\ 4)) \\
 \downarrow \\
 (2\ +\ (3\ *\ 4))
 \end{array}$$

Ayuda: Una expresión puede ser un número o una lista que contiene un operador y dos subexpresiones. Por ejemplo, para procesar:

$$(*\ (+\ 2\ 3)\ (-\ 8\ 4))$$

primero debe convertir recursivamente las expresiones $(+\ 2\ 3)$ y $(-\ 8\ 4)$, y posteriormente construir la expresión completa.

```
> (to-infix '(+ 2 (* 3 4)))
(2 + (3 * 4))
```

```
> (to-infix '(* (+ 2 3) (- 8 4)))
((2 + 3) * (8 - 4))
```

```
> (to-infix '(+ (* 2 5) 7))
((2 * 5) + 7)
```

Uso Ético de Inteligencia Artificial

Usos permitidos de IA: se permite utilizar herramientas de IA únicamente para:

1. Aclarar conceptos teóricos.
2. Solicitar explicaciones adicionales sobre el tema de estudio.
3. Pedir ejemplos ilustrativos distintos a los del taller.
4. Revisar estilo, redacción de documentación o comentarios.
5. Comprender mensajes de error del intérprete.

Usos NO permitidos de IA:

1. Solicitar a la IA la solución completa o parcial de los ejercicios.
2. Copiar y pegar código generado por IA sin comprensión.
3. Modificar mínimamente una solución generada por IA y presentarla como propia.
4. Usar IA durante una posible sustentación oral sin autorización.

Declaración obligatoria de uso de IA: si se utilizó IA en alguna parte del proceso, el grupo debe incluir al inicio del archivo:

;; Declaración de uso de IA:

;; Se utilizó IA para: (describir brevemente el uso)

;; Se declara que todos los integrantes comprenden y pueden explicar

;; completamente cada solución implementada.

Sustentación obligatoria: el Profesor podrá:

1. Solicitar sustentación individual o grupal de cualquier ejercicio.
2. Pedir modificación en vivo de una función.

3. Cambiar ejemplos de prueba.
4. Pedir reconstrucción parcial o total de algún algoritmo o funcionalidad.

Si un grupo/estudiante no puede explicar o defender su solución, la nota del ejercicio/taller será anulada (0.0), incluso si la solución es correcta y pasa las pruebas.

Criterios para detectar uso inadecuado de IA: Los siguientes criterios podrán considerarse señales de alerta; sin embargo, **no constituyen una lista exhaustiva ni limitativa**. El docente podrá apoyarse en otros elementos de juicio académico cuando lo considere pertinente:

1. Uso de estructuras o funciones no vistas en clase.
2. Estilos de programación inconsistentes con el nivel del curso.
3. Soluciones excesivamente complejas para el objetivo del ejercicio.
4. Código que los estudiantes no pueden explicar.
5. Similitud significativa entre entregas de distintos grupos.

Principio de responsabilidad académica El aprendizaje no puede delegarse en una herramienta tecnológica. Cuando la Inteligencia Artificial resuelve un problema en lugar del estudiante, se limita el desarrollo de las habilidades cognitivas que el curso busca fortalecer. En este sentido, el taller no pretende evaluar únicamente que el programa produzca un resultado correcto, sino que el estudiante demuestre comprensión del proceso de diseño, análisis y justificación formal que conduce a dicho resultado.

Consecuencias del uso indebido de IA: el uso indebido de IA podrá implicar:

1. Anulación del ejercicio.
2. Anulación del taller completo.
3. Reporte por falta a la ética académica según reglamento institucional.

Aclaraciones

1. El taller es en grupos de mínimo dos (2) alumnos, máximo tres (4).
2. La solución del taller debe ser subida al campus virtual en la fecha especificada allí mismo. Se debe subir al campus virtual en el enlace correspondiente a este taller un archivo comprimido **.zip** que siga la convención *Código de Estudiante1-Código de Estudiante2-Código de Estudiante3-Taller1FLP20262.zip*. Este archivo debe contener el archivo **ejercicios-taller1.rkt** que contenga el desarrollo de los ejercicios.
3. En las primeras líneas del archivo **ejercicios-taller1.rkt** deben estar comentados los nombres y los códigos de los estudiantes participantes.
4. También deben documentar los procedimientos que hayan implementado como solución a los problemas, las expresiones BNF de las estructuras que se están utilizando, y de igual manera para las funciones auxiliares, con ejemplos de prueba (mínimo 2 pruebas). Por ejemplo, si se pide un procedimiento *remove-first* debe ir así:

```
;; remove-first :  
;; Propósito:  
;; S x L -> L' : Procedimiento que remueve la primera  
;; ocurrencia de un símbolo S en una lista de símbolos L.  
;;  
;;<lista> := ()  
;;      := (<valor-de-scheme> <lista>)  
  
(define remove-first  
  (lambda (s los)  
    (if (null? los)  
        '()  
        (if (eqv? (car los) s)  
            (cdr los)  
            (cons (car los)  
                  (remove-first s (cdr los)))))))  
  
;; Pruebas  
(remove-first 'a '(a b c))
```

```
(remove-first 'b '(e f g))  
(remove-first 'a4 '(c1 a4 c1 a4))  
(remove-first 'x '())
```

5. **Importante:** Recuerde mantener la nomenclatura de las funciones que se piden (no les cambie el nombre), así como el orden y la cantidad de parámetros, en caso de que considere necesario añadir un parámetro, recurra al uso de funciones auxiliares.
6. En caso de tener dudas, puede consultar con la monitora **Emily Nuñez** - **emily.nunez@correounivalle.edu.co** en el horario de atención concertado vía correo electrónico.

Restricciones de implementación:

- El estudiante deberá implementar **todas las funciones auxiliares** necesarias para resolver cada ejercicio.
- Está prohibido utilizar funciones predefinidas que realicen operaciones que puedan ser implementadas mediante recursión y las funciones básicas del lenguaje. Por ejemplo: `append`, `concat`, `filter`, `map`, `reverse`, `length`, `member`, `sort`, entre otras.
- La lista anterior no es exhaustiva. Si una función predefinida puede ser reemplazada por una función implementada por el estudiante mediante recursión, **deberá implementarse dicha función**.
- Se podrán utilizar las funciones primitivas necesarias para manipular las estructuras de datos, tales como `null?`, `equal?`, `integer?`, `list?`, `car`, `cdr` y `cons`, además de las operaciones aritméticas y estructuras de control básicas.
- No está permitido ocultar el uso de funciones prohibidas dentro de funciones auxiliares ni mediante bibliotecas o módulos externos.
- Una solución que utilice funciones prohibidas podrá producir el resultado correcto, pero **no será considerada una solución válida**.

Principio general: Si una operación puede ser implementada por el estudiante utilizando las funciones básicas autorizadas y recursión, deberá implementarla él mismo.

Entregas Tardías o por Otros Medios

1. Este taller **sólo se recibirá a través del campus virtual**. Adicionalmente, sólo se evaluarán los documentos solicitados en el punto 2 de la sección anterior. Cualquier otro tipo de correo o nota aclaratoria será descartado.
2. Las entregas tarde serán penalizadas así: (-1pt de la nota final obtenida) por cada hora de retraso o fracción. Por ejemplo, si usted realiza su entrega y el campus registra las 24:00 (i.e., 1min después de la hora de entrega), usted está incurriendo en la primer hora de retraso. Asegurese con mínimo dos horas de anticipación que el link de carga funciona correctamente toda vez que es posible incurrir en una entrega tardía debido a los tiempos de respuesta.